

Flutter Stateful Widget Lifecycle (Easy Explanation)

1. createState()

Jab widget pehli baar banta hai (constructor ke baad), to Flutter is method ko call karta hai taake State object banaye.

✦ Yeh ek baar hi call hota hai.

Example:

```
@override
MyHomePageState createState() => MyHomePageState();
```

💡 Socho tum `MyHomePage(title: 'Test')` likhte ho — to pehle `createState()` call hoga, taake widget ke liye state ban sake.

2. initState()

Jab state object create ho jata hai, aur widget tree mein insert hone wala hota hai — tab ye method chalti hai.

✦ Yahan tum **API call, animations ya controllers** initialize karte ho.

Example:

```
@override
void initState() {
  super.initState();
  print("initState called");
}
```

💡 Jaise tum koi timer set karna chahte ho app start hote hi — to yeh `initState` mein karoge.

🔗 3. `didChangeDependencies()`

Jab widget kisi **InheritedWidget** (like Theme, MediaQuery) pe depend karta hai, aur wo cheez change hoti hai — to ye method chalti hai.

📌 Ye bhi **`initState` ke turant baad** ek baar call hoti hai.

Example:

```
@override
void didChangeDependencies() {
  super.didChangeDependencies();
  print("Dependencies changed");
}
```

💡 Socho tum `Theme.of(context)` use kar rahe ho — agar theme badle to ye method chalegi.

🔗 4. `build()`

Ye sabse important method hai — yahi tumhara **UI return karta hai**.

📌 Jab bhi tum `setState()` lagate ho, ye dobara call hota hai.

Example:

```
@override
Widget build(BuildContext context) {
  return Text("Hello World");
}
```

💡 Tumhara pura UI yahin se dikhaya jata hai — aur jab bhi koi state change ho, ye method UI dobara banata hai.

📦 5. setState()

Jab tum chahte ho ke koi **state update ho aur UI refresh ho**, to is method ko call karte ho.

✂ Isse Flutter ko pata chalta hai ke **UI dobara build karna hai**.

Example:

```
onPressed: () {  
  setState(() {  
    counter++;  
  });  
}
```

💡 Socho button click par counter barhna hai — to tum setState() ke andar us value ko update karoge.

📦 6. didUpdateWidget()

Jab widget dobara build hota hai lekin **naya data aata hai** (props change hoti hain), to ye method chalegi.

✂ Tum old aur new values compare kar sakte ho yahan.

Example:

```
@override  
void didUpdateWidget(MyWidget oldWidget) {  
  super.didUpdateWidget(oldWidget);  
  if (oldWidget.title != widget.title) {  
    print("Title changed");  
  }  
}
```

```
}
```

💡 Socho parent widget ne title update kiya ho — tum check kar sakte ho ke pehla kya tha aur ab kya hai.

🚫 7. deactivate()

Jab widget tree se temporarily **remove hota hai** (lekin shayad wapis aaye), to ye method chalti hai.

Example:

```
@override
void deactivate() {
    super.deactivate();
    print("Widget deactivated");
}
```

💡 Tumhare widget kisi tab ke andar ho aur tab switch karo — to wo deactivate ho sakta hai.

✖ 8. dispose()

Jab widget permanently tree se **remove ho jata hai**, to ye method call hoti hai — taake resources free kiye ja saken.

✂ Yahan tum `controller.dispose()` waghera likhte ho.

Example:

```
@override
void dispose() {
    controller.dispose();
    super.dispose();
}
```

}

💡 Socho tumne animation ya text controller lagaya tha — jab widget destroy ho raha ho, to unko band karna zaroori hai.

9. reassemble()

Jab **hot reload** hoti hai (sirf development time pe), to ye method chalti hai.

Example:

```
@override
void reassemble() {
  super.reassemble();
  print("Hot reload happened");
}
```

💡 Iska production mein koi role nahi, bas development mein helpful hota hai.

Summary Table:

Method	Kab Chalta Hai	Purpose
createState()	Pehli dafa widget banate waqt	State object create karne ke liye
initState()	State object ban gaya, insert hone se pehle	Initialization (API call, controller)
didChangeDependencies()	Jab dependencies change hoti hain	Inherited widgets handle karne ke liye
build()	UI banana	Widget tree return karta hai
setState()	Jab state update karni ho	UI rebuild trigger karta hai
didUpdateWidget()	Jab props change hoti hain	Old vs new widget props compare karte hain
deactivate()	Jab temporary remove ho	Tab switch ya re-ordering mein
dispose()	Jab permanently remove ho	Resources cleanup

reassemble()

Hot reload pe

Development time only